

Course: Advanced Databases

Group: A1

Team Members: Tobiasz Bazan, Jakub Kamiński, Adam Ćwikliński

Assignment: Idexes 2

1. Objective

The objective of the **Idexes 2** phase is to physically implement indexing strategies and measure their impact on the database workload.

Following feedback from the initial indexing proposal, our strategy was adjusted to target fields **without existing constraints** (avoiding Primary Keys).

To validate the effectiveness of these indexes, the workload benchmark was re-run under the exact identical conditions established in **Testing application** phase.

2. Implemented Indexing Strategy

To optimize the operations (specifically targeting Q5 and filtering queries), we implemented the following non-constraint indexes:

2.1 B-tree Indexes

Index on Activities.runner_id

```
SQL
CREATE INDEX idx_activities_runner_id ON Activities(runner_id);
```

Index on Telemetry_Data.activity_id

```
SQL
CREATE INDEX idx_telemetry_activity_id ON Telemetry_Data(activity_id);
```

2.2 Bitmap Index

Index on Activities.activity_type

```
SQL
CREATE BITMAP INDEX idx_activities_type ON Activities(activity_type);
```

2.3 Function-Based Index (Derived Data Optimization)

Index on extracted year from activity_date

```
SQL
CREATE INDEX idx_activity_year ON Activities(EXTRACT(YEAR FROM activity_date));
```

3. Execution Times Comparison

The table below compares the baseline average execution times (captured without indexes in the **Testing application** phase) against the new execution times with the proposed indexes active.

Note: All times are recorded in seconds.

3.1 B-Tree Indexes

Transaction	Scenario	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max	Avg
Q1	Without Indexes	1.557	1.421	1.405	1.416	1.408	1.405	1.557	1.441
	With B-Tree	1.032	0.948	0.961	1.607	1.271	0.948	1.607	1.164
Q2	Without Index	1.046	1.017	1.004	1.001	1.005	1.004	1.046	1.015

	With B-Tree	0.980	0.881	0.796	0.878	1.116	0.796	1.116	0.930
Q3	Without Index	0.579	0.576	0.570	0.563	0.569	0.563	0.579	0.571
	With B-Tree	0.745	0.578	0.582	0.678	0.740	0.578	0.745	0.665
Q4 (INSERT)	Without Index	0.238	0.206	0.202	0.205	0.204	0.202	0.238	0.211
	With B-Tree	0.220	0.204	0.197	0.207	0.235	0.197	0.235	0.213
Q5 (UPDATE)	Without Index	5.822	5.818	5.861	5.820	5.861	5.818	5.861	5.836
	With B-Tree	0.326	0.331	0.240	0.358	0.386	0.240	0.386	0.328
Q6 (DELETE)	Without Index	1.112	0.758	1.259	1.699	1.310	0.758	1.699	1.228
	With B-Tree	0.996	1.497	1.982	3.399	3.097	0.996	3.399	2.194
Whole Workload	Without Index	10.35	9.796	10.30	10.704	10.357	9.796	10.704	10.302
	With B-Tree	4.299	4.439	4.758	7.127	6.845	4.299	7.127	5.494

- The B-Tree indexes successfully eliminated the primary bottleneck, reducing Q5's average execution time by 94% (from 5.836s to 0.328s). However, Q6 experienced a performance degradation, indicating index maintenance overhead for that specific DELETE operation.

3.2 Bitmap Index

Transaction	Scenario	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max	Avg
Q1	Without Index	1.557	1.421	1.405	1.416	1.408	1.405	1.557	1.441
	With Bitmap	0.576	0.649	0.623	0.875	0.625	0.576	0.875	0.670
Q2	Without Index	1.046	1.017	1.004	1.001	1.005	1.004	1.046	1.015
	With Bitmap	0.560	0.540	0.673	0.960	0.565	0.540	0.960	0.660
Q3	Without Index	0.579	0.576	0.570	0.563	0.569	0.563	0.579	0.571
	With Bitmap	0.562	0.533	0.537	0.811	0.548	0.533	0.811	0.598
Q4 (INSERT)	Without Index	0.238	0.206	0.202	0.205	0.204	0.202	0.238	0.211
	With Bitmap	0.206	0.308	0.196	0.303	0.203	0.196	0.308	0.243
Q5 (UPDATE)	Without Index	5.822	5.818	5.861	5.820	5.861	5.818	5.861	5.836
	With Bitmap	5.993	7.083	6.751	6.748	5.953	5.953	7.083	6.506

Q6 (DELETE)	Without Index	1.112	0.758	1.259	1.699	1.310	0.758	1.699	1.228
	With Bitmap	2.276	3.664	3.690	3.712	2.533	2.276	3.712	3.175
Whole Workload	Without Index	10.354	9.796	10.301	10.704	10.357	9.796	10.704	10.302
	With Bitmap	10.173	12.777	12.470	13.409	10.427	10.173	13.409	11.851

- The Bitmap index optimized read-heavy filtering, cutting Q1's average execution time in half (from 1.441s to 0.670s). On the other hand, it caused performance degradation in Q5 and Q6 transactions due to the heavy locking overhead required to update bitmap arrays during data manipulation.

3.3 Function-Based Index (FBI)

Transaction	Scenario	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max	Avg
Q1	Without Index	1.557	1.421	1.405	1.416	1.408	1.405	1.557	1.441
	With FBI	0.670	0.659	0.668	0.621	0.787	0.621	0.787	0.681
Q2	Without Index	1.046	1.017	1.004	1.001	1.005	1.004	1.046	1.015
	With FBI	0.717	0.870	0.791	0.698	0.707	0.698	0.870	0.757
Q3	Without Index	0.579	0.576	0.570	0.563	0.569	0.563	0.579	0.571
	With FBI	0.606	0.591	0.621	0.575	0.581	0.575	0.621	0.595
Q4 (INSERT)	Without Index	0.238	0.206	0.202	0.205	0.204	0.202	0.238	0.211
	With FBI	0.213	0.203	0.292	0.204	0.202	0.202	0.292	0.223
Q5 (UPDATE)	Without Index	5.822	5.818	5.861	5.820	5.861	5.818	5.861	5.836
	With FBI	5.962	5.855	6.834	5.893	5.975	5.855	6.834	6.104

Q6 (DELETE)	Without Index	1.112	0.758	1.259	1.699	1.310	0.758	1.699	1.228
	With FBI	3.326	2.124	1.502	2.542	1.950	1.502	3.326	2.289
Whole Workload	Without Index	10.354	9.796	10.301	10.704	10.357	9.796	10.704	10.302
	With FBI	11.494	10.302	10.708	10.533	10.202	10.202	11.49	10.64

- The Function-Based Index improved the analytical SELECT queries (Q1 and Q2) by pre-calculating the date extraction. It introduced minor maintenance overhead for DML operations and did not resolve the primary Q5 bottleneck.

3.4 All Indexes Combined

Transaction	Scenario	Run 1	Run 2	Run 3	Run 4	Run 5	Min	Max	Avg
Q1	Without Index	1.557	1.421	1.405	1.416	1.408	1.405	1.557	1.441
	All Indexes	1.137	1.085	1.062	1.057	1.080	1.057	1.137	1.084
Q2	Without Index	1.046	1.017	1.004	1.001	1.005	1.004	1.046	1.015
	All Indexes	1.008	1.386	1.000	1.058	1.001	1.000	1.386	1.091
Q3	Without Index	0.579	0.576	0.570	0.563	0.569	0.563	0.579	0.571
	All Indexes	0.732	0.706	0.641	0.636	0.623	0.623	0.732	0.668
Q4 (INSERT)	Without Index	0.238	0.206	0.202	0.205	0.204	0.202	0.238	0.211
	All Indexes	0.331	0.208	0.209	0.211	0.207	0.207	0.331	0.233
Q5 (UPDATE)	Without Index	5.822	5.818	5.861	5.820	5.861	5.818	5.861	5.836
	All Indexes	0.385	0.247	0.259	0.247	0.244	0.244	0.385	0.276
Q6 (DELETE)	Without Index	1.112	0.758	1.259	1.699	1.310	0.758	1.699	1.228

	All Indexes	1.737	2.916	0.853	1.153	1.034	0.853	2.916	1.539
Whole Workload	Without Index	10.354	9.796	10.301	10.704	10.357	9.796	10.704	10.302
	All Indexes	5.330	6.548	4.024	4.362	4.189	4.024	6.548	4.891

- Combining all indexes allowed the optimizer to leverage the strengths of each architecture. The overall workload execution time was reduced by over 50% (from 10.302s to 4.891s), fully resolving the Q5 bottleneck while maintaining optimized read performance

4. Experimental Analysis and Conclusions

4.1 Overall Performance Impact The implementation of the proposed indexing strategy was highly successful. The total workload execution time was reduced by more than 50%, dropping from a baseline average of 10.302 seconds to 4.891 seconds.

4.2 Elimination of the Primary Bottleneck The B-Tree index on Activities(activity_date) successfully resolved the primary bottleneck identified in Phase 7. Query 5 (UPDATE) execution time was reduced by 95%, from 5.836 seconds to 0.276 seconds, confirming a successful shift from a TABLE ACCESS FULL to an INDEX RANGE SCAN.

4.3 Query Plan Comparison (Query 5: UPDATE) To prove the physical optimization, we compared the execution plans for Q5 before and after the indexes were applied.

Before (Baseline Phase 7): Before indexing, the database had to perform a massive TABLE ACCESS FULL on the ACTIVITIES table, resulting in a total computational Cost of 11 million.

```

-----
| Id | Operation          | Name          | Rows | Bytes | Cost (%CPU)| Time   |
-----
| 0 | UPDATE STATEMENT    |               | 1800 | 64800 | 11M (1)| 00:07:46 |
| 1 | UPDATE              | TRAINING_GOALS |      |      |      |      |
|* 2 | HASH JOIN RIGHT SEMI |              | 1800 | 64800 | 120 (4)| 00:00:01 |
| 3 | VIEW                | VW_NSO_2      | 1200 | 15600 | 111 (4)| 00:00:01 |
|* 4 | HASH JOIN RIGHT SEMI |              | 1200 | 45600 | 111 (4)| 00:00:01 |
|* 5 | TABLE ACCESS FULL  | RUNNERS       | 1200 | 14400 | 5 (0)| 00:00:01 |
| 6 | VIEW                | VW_SQ_1       | 1200 | 31200 | 105 (3)| 00:00:01 |
|* 7 | SORT GROUP BY       |              | 1200 | 9600  | 105 (3)| 00:00:01 |
| 8 | TABLE ACCESS FULL  | ACTIVITIES    | 36000 | 281K | 103 (1)| 00:00:01 |
-----

```

After (Optimized Phase 9): After indexing, the EXPLAIN PLAN confirms the database successfully shifted to an INDEX RANGE SCAN using IDX_ACTIVITIES_RUNNER_ID. This bypassed the full table scans and dropped the total computational Cost from 11M down to 737K.

```

-----
| Id | Operation          | Name          | Rows | Bytes | Cost (%CPU)| Time   |
-----
| 0 | UPDATE STATEMENT    |               | 90   | 2250 | 737K (1)| 00:00:29 |
| 1 | UPDATE              | TRAINING_GOALS |      |      |      |      |
| 2 | NESTED LOOPS SEMI   |              | 90   | 2250 | 57621 (1)| 00:00:03 |
| 3 | TABLE ACCESS FULL  | TRAINING_GOALS | 1800 | 41400 | 9 (0)| 00:00:01 |
| 4 | VIEW PUSHED PREDICATE | VW_NSO_2      | 1    | 2    | 32 (0)| 00:00:01 |
| 5 | NESTED LOOPS        |              | 1    | 38   | 32 (0)| 00:00:01 |
|* 6 | TABLE ACCESS BY INDEX ROWID | RUNNERS       | 1    | 12   | 2 (0)| 00:00:01 |
|* 7 | INDEX UNIQUE SCAN    | SYS_C008255    | 1    |      | 1 (0)| 00:00:01 |
| 8 | VIEW                | VW_SQ_1       | 1    | 26   | 30 (0)| 00:00:01 |
|* 9 | SORT GROUP BY       |              | 1    | 8    | 30 (0)| 00:00:01 |
| 10 | TABLE ACCESS BY INDEX ROWID BATCHED | ACTIVITIES    | 30   | 240  | 30 (0)| 00:00:01 |
|* 11 | INDEX RANGE SCAN     | IDX_ACTIVITIES_RUNNER_ID | 30   |      | 1 (0)| 00:00:01 |
-----

```

Conclusion of Plans: The database recognized the new B-Tree index structure and successfully transformed the execution path, reducing the mathematical query cost by roughly 93% and resulting in the massive real-world time savings documented in Section 3

Final Conclusion: Providing the database with a combination of B-Tree, Bitmap, and Function-Based indexes allowed it to dynamically select the most efficient access paths for both read and write operations. The massive gains in SELECT and UPDATE queries offset the minor overhead of INSERT, validating the complete optimization strategy.